

Deep Reinforcement Learning — Assignment 2

Ido Raviel & Dan Butensky

Introduction

This report documents our implementation of Soft Actor-Critic (SAC) applied to two continuous-control MuJoCo environments: Reacher-v4 and Ant-v4. We implemented the algorithm from scratch in PyTorch with no external RL libraries. SAC is an off-policy actor-critic algorithm designed for continuous action spaces. It maximizes a trade-off between expected return and entropy, encouraging the agent to explore broadly while still learning a high-performing policy. Key features include twin Q-critics with Polyak-averaged target networks, a tanh-squashed Gaussian policy with reparameterized sampling, and automatic entropy temperature tuning (learned α).



Environments

We trained our SAC agent on two MuJoCo v4 environments, chosen to cover different levels of complexity — a short-horizon reaching task and a high-dimensional locomotion task.

Reacher-v4

Reacher-v4 is a simple 2-DOF robotic arm task. The agent controls two joints to move the fingertip as close as possible to a randomly placed target. The observation space is 11-dimensional (joint angles, velocities, and target position) and the action space is 2-dimensional (torques), bounded to $[-1, 1]$. Episodes last 50 steps. The reward is negative distance to the target, so higher (less negative) is better, with a theoretical maximum near 0.

Ant-v4

Ant-v4 is a challenging locomotion task where a four-legged ant robot must learn to walk as fast as possible. The observation space is 27-dimensional and the action space is 8-dimensional (joint torques), bounded to $[-1, 1]$. Episodes last up to 1000 steps. The reward combines forward velocity, a survival bonus, and penalties for excessive control effort. This is a significantly harder task than Reacher, requiring the agent to coordinate multiple limbs over a long horizon.

Environment	Obs. Dim	Act. Dim	Action Range	Episode Steps
Reacher-v4	11	2	$[-1, 1]$	50
Ant-v4	27	8	$[-1, 1]$	1000

Algorithm — Soft Actor-Critic (SAC)

SAC is an off-policy maximum-entropy reinforcement learning algorithm. Rather than maximizing expected return alone, SAC augments the objective with an entropy bonus, encouraging the agent to maintain a stochastic policy throughout training. This improves exploration and robustness. Our

implementation consists of the following components:

- **Twin Q-critics:** Two independent Q-networks $Q_1(s, a)$ and $Q_2(s, a)$ are trained in parallel. The minimum of the two is used for policy and target computation, reducing overestimation bias (clipped double-Q). Each critic has a corresponding frozen target network updated via Polyak averaging ($\tau = 0.005$).
- **Tanh-squashed Gaussian policy:** The actor outputs a state-dependent mean and log-std, samples via the reparameterization trick, and squashes through tanh to guarantee actions remain within bounds. The log-probability is corrected for the tanh transformation to give an exact entropy estimate.
- **Automatic entropy tuning:** The temperature parameter α , which controls the entropy-reward trade-off, is learned automatically by optimizing a dual objective that drives the policy entropy toward a target value. The target entropy is set to $-\dim(\text{action space})$.
- **Update order per step:** Critics $\rightarrow$ actor $\rightarrow$ temperature $\rightarrow$ Polyak target update. One gradient update is performed per environment step.

Network Architecture

All networks (actor and both critics) share the same MLP architecture: two hidden layers of 256 units each with ReLU activations. The actor has a shared trunk with separate mean and log-std heads; the critics concatenate state and action as input.

Network	Input	Hidden Layers	Output
Actor (GaussianPolicy)	obs_dim	256, 256 (ReLU)	mean + log_std (act_dim each)
Critic $\times 2$ (QNetwork)	obs_dim + act_dim	256, 256 (ReLU)	scalar Q-value
Target Critics $\times 2$	obs_dim + act_dim	256, 256 (ReLU)	scalar Q-value (frozen)

Hyperparameters

Shared hyperparameters are listed first, followed by per-environment overrides.

Shared Hyperparameters

Parameter	Value
Optimizer	Adam (actor, critics, temperature)
Learning rate	3e-4
Discount factor (γ)	0.99
Polyak coefficient (τ)	0.005
Batch size	256
Replay buffer capacity	1M
Hidden layer sizes	[256, 256]
Log-std clamp	[-20, 2]
Initial temperature (alpha_0)	0.2
Automatic entropy tuning	Yes
Target entropy	$-\dim(\text{action space})$
Updates per env step	1
Evaluation frequency	Every 10K steps
Evaluation episodes (training)	10
Evaluation episodes (final)	100

Per-Environment Overrides

Parameter	Reacher-v4	Ant-v4
Total steps	150K	5M
Warmup steps	5K	30K

What Didn't Work

Too Few Training Steps for Ant-v4

Our initial attempt trained Ant-v4 for only 1M steps — the same budget we considered for simpler environments. The agent showed early signs of learning (rewards climbing to ~2000–3000) but had clearly not converged by the end of training. Ant requires the agent to coordinate 8 joints over 1000-step episodes, which demands significantly more environment interaction than short-horizon tasks. Extending to 5M steps resolved this and allowed the agent to reach rewards in the 6000–7000 range.

Insufficient Warmup for Ant-v4

With the default warmup of 5K random steps (used for Reacher), Ant training was unstable in the early phase — the replay buffer contained too little diversity for meaningful critic updates on such a complex observation space. Increasing the warmup to 30K random steps before gradient updates begin stabilized early training and improved the consistency of learning across seeds.

What Worked

SAC Hyperparameters Worked Well Out of the Box

The most notable finding of our experiments is that SAC required almost no tuning to perform well on both environments. The hyperparameters we chose — Adam with $\text{lr} = 3\text{e-}4$, hidden sizes [256, 256], $\gamma = 0.99$, $\tau = 0.005$, batch size 256 — worked reliably across both tasks without modification. This reflects SAC’s robustness as an algorithm: the entropy regularization naturally handles exploration, the automatic temperature tuning adapts to each environment’s action dimensionality, and the twin-critic architecture keeps value estimates stable.

Automatic Entropy Tuning

Setting the target entropy to $-\text{dim}(\text{action space})$ worked well for both environments without any manual adjustment. For Reacher (2-dim actions) this drives the policy toward a target entropy of -2 , and for Ant (8-dim actions) toward -8 . The learned α adapted automatically during training, starting high to encourage exploration and gradually decreasing as the policy converged. This removed an entire hyperparameter from the tuning process.

Separate terminated / truncated Handling

We correctly separated the environment’s ‘terminated’ flag (true terminal states, e.g. the ant falling over) from ‘truncated’ (time-limit cutoffs). Only ‘terminated’ is used as the done signal in the Bellman update, so the agent correctly bootstraps value past artificial episode endings. This is especially important for Ant, where the 1000-step limit is frequently hit by a healthy agent, and treating truncation as termination would incorrectly penalize long survival.

Results

Each environment was trained for 3 independent seeds. During training, evaluation was performed every 10K steps by averaging 10 deterministic episodes ($\epsilon=0$, mean action). Final reported results are scored over 100 deterministic episodes on the best saved checkpoint per seed.

Reacher-v4

Reacher converges quickly — all three seeds reach near-optimal performance by $\sim 20\text{K}$ steps and plateau around -4.1 by 150K steps. The reward in Reacher is negative Euclidean distance to the target, so values close to 0 indicate near-perfect tracking. All seeds are highly consistent, confirming that SAC is stable on this task.

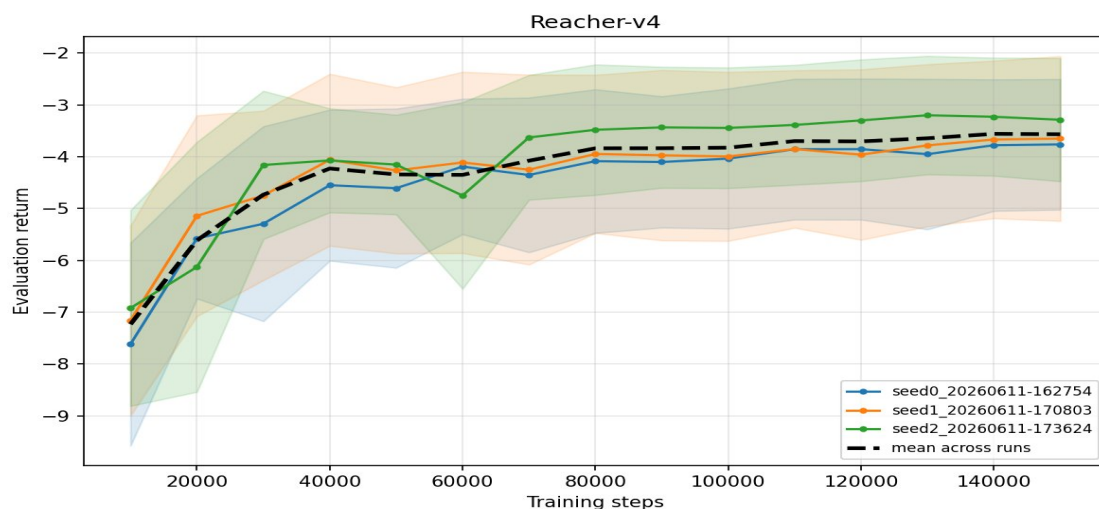


Figure 1: Evaluation return across all 3 seeds for Reacher-v4 (shaded = ± 1 std over 10 eval episodes).

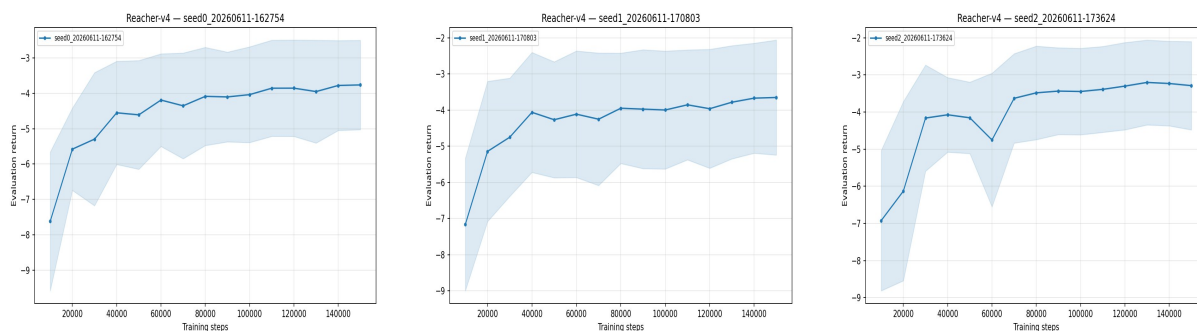


Figure 2: Individual learning curves for Reacher-v4 — seed 0, seed 1, seed 2.

Final results over 100 evaluation episodes (best checkpoint per seed):

	Seed 0	Seed 1	Seed 2	Average
Mean $\pm$ Std	-4.11 ± 1.67	-4.10 ± 1.58	-4.06 ± 1.62	-4.09

Ant-v4

Ant is a significantly harder task. All three seeds show a consistent learning pattern: rapid improvement from 0 to ~ 1 M steps, reaching rewards around 5000–6000, followed by slower refinement toward 6000–7000 by 5M steps. The occasional sharp drops in the learning curves likely reflect sampling unlucky batches of transitions during the 10K steps between evaluations — a temporary degradation in the policy that the agent recovered from quickly in subsequent updates. The final 100-episode evaluation confirms robust performance well above 6000 across all seeds.

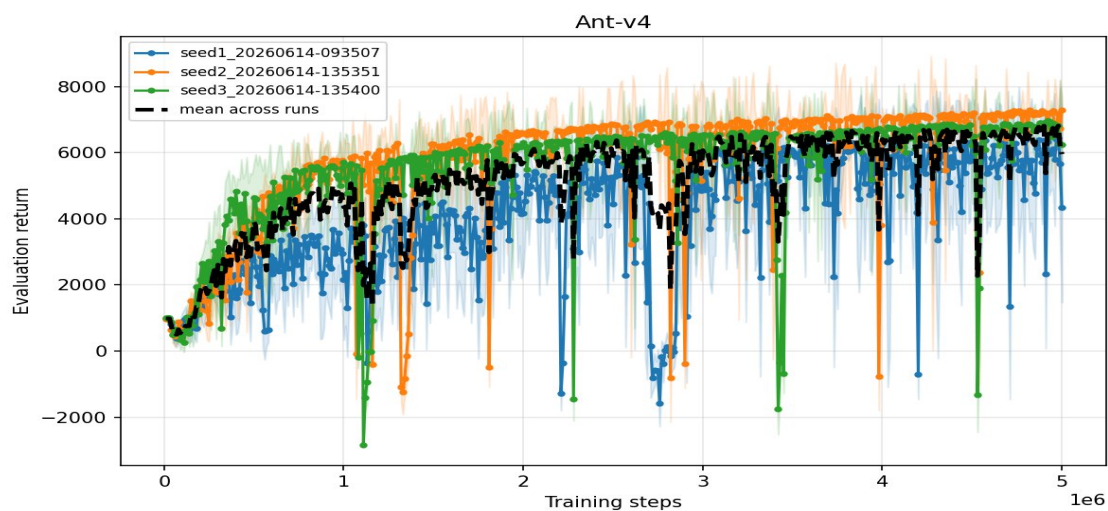


Figure 3: Evaluation return across all 3 seeds for Ant-v4 (shaded = ± 1 std over 10 eval episodes).

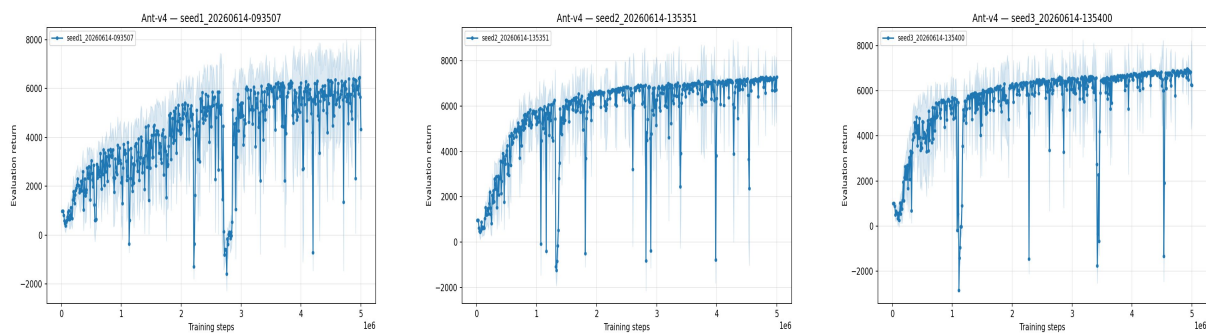


Figure 4: Individual learning curves for Ant-v4 — seed 1, seed 2, seed 3.

Final results over 100 evaluation episodes (best checkpoint per seed):

	Seed 1	Seed 2	Seed 3	Average
Mean $\pm$ Std	6314.91 $\pm$ 972.33	7113.52 $\pm$ 1024.88	6757.49 $\pm$ 974.99	6728.64